BAHAN AJAR

KOLABORASI GITHUB

Membuat Akun GitHub, Menghubungkan Repositori Lokal, dan Mengunggah Kode Website Agensi

Satuan Pendidikan	SMK Negeri 2 Banjarmasin
Mata Pelajaran	Informatika / PPLG - Sistem Operasi dan Tools
Kelas / Fase / Semester	X PPLG / Fase E / I (Ganjil)
Alokasi Waktu	1 pertemuan (2 JP x 45 menit / 90 menit)
Proyek Praktik	Website Agensi Game - web-agensi-game

Disusun berdasarkan Modul Ajar Pertemuan Minggu ke-5
Tahun Ajaran 2026/2027

Penyusun modul: Allip Ma’rup Syahbana

IDENTITAS DAN PANDUAN PENGGUNAAN

Komponen	Keterangan
Topik	Kolaborasi GitHub: Membuat Akun GitHub dan Push Kode Web Agensi ke Repositori
Sasaran	Peserta didik Kelas X PPLG, Fase E
Prasyarat	Memiliki folder proyek lokal web-agensi-game, sudah menjalankan git init dan minimal satu git commit, mengenal Terminal, serta memiliki email aktif.
Perangkat	Komputer/laptop dengan VM Ubuntu, Git terpasang, browser, koneksi internet, dan proyektor untuk demonstrasi.
Produk akhir	Repositori GitHub web-agensi-game berisi index.html dan riwayat commit, perubahan lanjutan yang berhasil di-push, serta kolaborator yang telah menerima undangan.

Tujuan Pembelajaran

- Peserta didik dapat menjelaskan pengertian GitHub, perbedaan Git dan GitHub, serta fungsi GitHub sebagai remote repository.
- Peserta didik dapat membuat akun GitHub dan repositori web-agensi-game dengan pengaturan sesuai instruksi.
- Peserta didik dapat membuat Personal Access Token (PAT), menggunakannya untuk autentikasi push melalui HTTPS, dan menjaga kerahasiaannya.
- Peserta didik dapat menghubungkan repositori lokal dengan remote GitHub menggunakan git remote add origin dan memeriksanya dengan git remote -v.
- Peserta didik dapat melakukan push pertama dan push lanjutan, serta memeriksa hasilnya di GitHub.
- Peserta didik dapat menambahkan anggota kelompok sebagai kolaborator dan menerapkan alur kerja pull-edit-add-commit-push.

Peta Materi

Urutan	Materi yang Dipelajari	Hasil yang Diharapkan
1	Konsep Git, GitHub, repositori lokal dan remote	Memahami fungsi masing-masing komponen.
2	Membuat akun dan repositori GitHub	Akun terverifikasi dan repo web-agensi-game tersedia.
3	Membuat Personal Access Token	PAT tersedia dan tersimpan aman.
4	Menghubungkan remote dan push pertama	Commit lokal tampil di GitHub.

Urutan	Materi yang Dipelajari	Hasil yang Diharapkan
5	Perubahan lanjutan dan kolaborator	Commit baru terkirim dan undangan diterima.
6	Alur kolaborasi dan troubleshooting	Mampu bekerja terkoordinasi dan mengatasi kendala dasar.

CARA MENGGUNAKAN BAHAN AJAR Baca materi secara berurutan. Ikuti demonstrasi guru, lalu praktikkan setiap perintah pada folder proyek web-agensi-game milik sendiri. Jangan menjalankan perintah sebelum memastikan Terminal berada di folder proyek yang benar.

1. MENGENAL VERSION CONTROL, GIT, DAN GITHUB

1.1 Apa itu Version Control?

Version Control System (VCS) atau sistem pengendali versi adalah cara untuk mencatat perubahan pada file proyek dari waktu ke waktu. Dalam pengembangan website, file seperti index.html, style.css, dan script.js dapat berubah berkali-kali. Version control membantu developer mengetahui perubahan yang pernah dibuat, menyimpan titik-titik versi, dan menelusuri kembali riwayat proyek.

Bayangkan seorang siswa menulis laporan. Ia menyimpan beberapa salinan: laporan-awal.docx, laporan-revisi.docx, laporan-final.docx, dan laporan-final-baru.docx. Cara ini mudah membingungkan karena nama file menumpuk. Version control menyimpan riwayat perubahan secara lebih teratur dalam satu proyek.

Manfaat version control

- Mencatat riwayat perubahan sehingga developer dapat melihat kapan dan apa yang berubah.
- Membantu mengembalikan atau membandingkan versi sebelumnya ketika terjadi kesalahan.
- Memudahkan beberapa orang bekerja pada proyek yang sama.
- Membantu mengetahui perubahan yang telah disimpan sebagai commit.

1.2 Apa itu Git?

Git adalah perangkat lunak version control yang dijalankan pada komputer atau VM. Git mencatat perubahan proyek dan menyimpannya dalam bentuk riwayat commit di repositori lokal. Git dapat digunakan tanpa akun GitHub dan tanpa koneksi internet untuk pekerjaan yang hanya dilakukan pada komputer sendiri.

1.3 Apa itu GitHub?

GitHub adalah platform hosting berbasis cloud untuk menyimpan repositori Git secara online. Melalui GitHub, developer dapat mengunggah kode, membagikan proyek, mengundang kolaborator, melihat riwayat perubahan, serta menyiapkan proyek untuk dipublikasikan melalui GitHub Pages.

ANALOGI SEDERHANA Git adalah alat pencatat riwayat pada buku catatan proyek. Repositori lokal adalah buku yang disimpan di komputer sendiri. GitHub adalah tempat penyimpanan online

> tempat salinan repositori dapat dibagikan kepada anggota tim.

1.4 Perbedaan Git dan GitHub

Aspek	Git	GitHub
Bentuk	Software / version control system.	Platform hosting repositori berbasis cloud.
Tempat digunakan	Di komputer atau VM sendiri.	Melalui internet dan browser; dapat diakses dengan Git dari Terminal.
Fungsi utama	Mencatat dan mengelola riwayat perubahan proyek.	Menyimpan repositori secara online dan mendukung kolaborasi.
Koneksi internet	Dapat digunakan secara lokal tanpa internet.	Memerlukan internet untuk berinteraksi dengan layanan GitHub.
Hubungan	Dapat digunakan tanpa GitHub.	Menyimpan dan mengelola repositori yang menggunakan Git.

1.5 Istilah penting

Istilah	Penjelasan sederhana
Repository / repo	Folder proyek yang dikelola Git. Di dalamnya terdapat file proyek dan folder tersembunyi .git yang menyimpan data Git.
Local repository	Repositori yang berada di komputer atau VM milik pengguna.
Remote repository	Repositori yang berada di server/layanan online, dalam materi ini GitHub.
Commit	Catatan/snapshot perubahan yang disimpan ke riwayat repositori lokal.
Branch	Jalur pengembangan yang memungkinkan pekerjaan dilakukan pada cabang tertentu. Pada latihan ini nama branch utama adalah main.
Push	Mengirim commit dari repositori lokal ke repositori remote.
Pull	Mengambil sekaligus menggabungkan perubahan terbaru dari remote ke repositori lokal.
Collaborator	Pengguna GitHub yang diberi izin untuk mengakses dan berkontribusi pada repositori.

2. MENGAPA GITHUB PENTING UNTUK KOLABORASI?

2.1 Manfaat GitHub

Manfaat	Penjelasan dan contoh
Backup di cloud	Kode proyek tidak hanya tersimpan di laptop/VM. Jika perangkat lokal bermasalah, salinan yang sudah di-push masih tersedia di GitHub.
Kolaborasi tim	Anggota tim dapat mengambil perubahan terbaru dengan git pull dan mengirim perubahan dengan git push sesuai izin akses.
Portofolio developer	Repositori dapat menjadi bukti proyek yang pernah dibuat dan kemampuan menggunakan tools pengembangan.
Persiapan publikasi	Repositori GitHub yang berisi kode menjadi sumber untuk mempublikasikan website melalui GitHub Pages pada pertemuan selanjutnya.

2.2 Memahami alur lokal dan remote

Arah	Perintah	Apa yang terjadi?
Lokal ke GitHub	git push	Commit yang sudah tersimpan di lokal dikirim ke remote GitHub.
GitHub ke lokal	git pull	Perubahan terbaru dari remote diambil dan digabungkan ke repositori lokal.

INGAT git commit belum mengunggah kode ke internet. Commit hanya menyimpan perubahan pada repositori lokal. Agar perubahan tampil di GitHub, lakukan git push.

2.3 Alur kerja yang akan dilakukan

Tahap	Aktivitas	Lokasi hasil
Persiapan	Membuat akun dan repositori GitHub.	Akun dan server GitHub.
Penghubungan	Menambahkan alamat remote origin ke repo lokal.	Konfigurasi repo lokal.
Push pertama	Mengirim branch main ke GitHub.	Repo online.
Pembaruan	Edit file, git add, git commit, lalu git push.	Commit terbaru pada repo online.
Kolaborasi	Mengundang anggota dan menerima undangan.	Akses anggota pada repo.

Tahap	Aktivitas	Lokasi hasil
Kerja harian	git pull sebelum bekerja, kemudian edit, add, commit, dan push.	Repo tim tetap sinkron.

3. MEMBUAT AKUN GITHUB

Setiap peserta didik membuat akun sendiri. Akun tersebut akan digunakan untuk menyimpan proyek, mengelola repositori, dan berkolaborasi. Gunakan username yang profesional karena username dapat menjadi bagian dari alamat repositori dan portofolio developer.

3.1 Persiapan

- Siapkan alamat email aktif yang dapat dibuka saat pendaftaran.
- Siapkan username yang mudah dikenali dan tetap profesional.
- Gunakan kata sandi yang kuat dan tidak sama dengan kata sandi akun lain.
- Pastikan komputer terhubung ke internet dan browser dapat dibuka.

3.2 Langkah membuat akun

1. Buka browser, kemudian kunjungi https://github.com/.
2. Pilih Sign up untuk memulai pendaftaran akun.
3. Masukkan alamat email yang aktif dan ikuti instruksi pada layar.
4. Buat kata sandi yang kuat sesuai persyaratan yang ditampilkan.
5. Tentukan username yang profesional. Contoh pola: nama, nama-belakang, atau variasi nama yang mudah dikenali dan belum digunakan.
6. Ikuti langkah verifikasi atau konfirmasi keamanan yang diminta GitHub.
7. Buka email dari GitHub, lalu lakukan verifikasi email sesuai petunjuk.
8. Setelah akun aktif, masuk (sign in) ke GitHub dan pastikan profil dapat dibuka.

3.3 Memilih username yang baik

Disarankan	Hindari
Menggunakan nama yang mudah dikenali.	Nama yang sulit dibaca atau terlalu banyak karakter acak.
Singkat, konsisten, dan sopan.	Kata-kata tidak profesional atau menyinggung.
Dapat digunakan sebagai identitas portofolio.	Memasukkan informasi pribadi yang tidak perlu.

PEMERIKSAAN Akun siap digunakan apabila peserta didik berhasil masuk ke GitHub dan email telah diverifikasi. Catat username sendiri pada LKPD, tetapi jangan pernah mencatat atau membagikan kata sandi.

4. MEMBUAT REPOSITORI WEB-AGENSI-GAME

Repositori adalah tempat penyimpanan proyek yang dikelola Git. Pada latihan ini, nama repositori harus sama dengan nama proyek lokal agar mudah dikenali, yaitu web-agensi-game.

4.1 Pengaturan repositori

Pengaturan	Nilai yang dipilih	Alasan
Repository name	web-agensi-game	Sesuai dengan folder proyek yang sudah dibuat sebelumnya.
Visibility	Public	Repositori dapat dilihat oleh publik dan digunakan sebagai portofolio serta sumber publikasi latihan.
Add a README file	Jangan dicentang	Repositori lokal sudah memiliki riwayat commit. Repo remote dibuat kosong agar push pertama tidak berbenturan dengan commit awal yang berbeda.

4.2 Langkah membuat repositori

1. Setelah masuk ke GitHub, buka halaman utama akun.
2. Pilih tombol New repository. Jika tombol tidak terlihat, gunakan menu pembuatan repositori yang tersedia pada tampilan akun.
3. Isi Repository name dengan web-agensi-game.
4. Pilih Public pada pengaturan visibility.
5. Pastikan pilihan Add a README file tidak dicentang. Jangan menambahkan .gitignore atau license untuk latihan ini kecuali guru menginstruksikan lain.
6. Tinjau kembali pengaturan, lalu klik Create repository.
7. Halaman repositori baru akan terbuka. Biarkan halaman ini terbuka karena alamat URL repositori akan digunakan pada langkah berikutnya.

Contoh pola alamat repositori (ganti USERNAME_GITHUB dengan username akun sendiri)

> https://github.com/USERNAME_GITHUB/web-agensi-game

> **PENTING** Jangan memasukkan file rahasia, kata sandi, Personal Access Token, atau data pribadi ke repositori Public. Semua file yang diunggah ke repositori publik berpotensi dapat dilihat orang lain.

5. PERSONAL ACCESS TOKEN (PAT)

5.1 Pengertian PAT

Personal Access Token (PAT) adalah token akses pribadi yang dapat digunakan sebagai kredensial saat melakukan operasi Git melalui HTTPS. Dalam latihan ini, PAT dimasukkan ketika Terminal meminta

Password pada proses git push. PAT bukan kata sandi akun biasa; token berfungsi sebagai bukti autentikasi untuk mengizinkan tindakan sesuai hak akses yang diberikan.

5.2 Mengapa PAT diperlukan?

GitHub tidak menerima kata sandi akun biasa untuk autentikasi operasi Git melalui HTTPS. Ketika Git meminta username dan password, masukkan username GitHub pada kolom Username dan PAT pada kolom Password. Ketika token diketik atau ditempel pada Terminal, karakter biasanya tidak ditampilkan. Hal ini normal dan bukan berarti Terminal macet.

5.3 Langkah membuat PAT sesuai latihan modul

> **CATATAN PEMBELAJARAN** Langkah praktik di bawah mengikuti modul ajar: Personal Access Token (classic) dengan scope repo. Jenis token dan tampilan menu dapat berubah mengikuti kebijakan GitHub. Jika opsi yang diajarkan tidak tersedia pada akun, hentikan langkah dan minta arahan guru; jangan memberi izin secara sembarangan.

1. Klik foto profil akun GitHub pada pojok kanan atas.
2. Pilih Settings.
3. Pada bagian pengaturan akun, buka Developer settings.
4. Pilih Personal access tokens, lalu Tokens (classic).
5. Pilih Generate new token, kemudian pilih opsi token classic jika diminta.
6. Isi Note atau nama token, misalnya PAT-Praktik-X-PPLG, agar tujuan token mudah dikenali.
7. Tentukan masa berlaku token sesuai arahan guru dan pilihan yang tersedia.
8. Pada bagian Select scopes, centang repo sesuai modul ajar untuk memberikan akses yang diperlukan pada repositori.
9. Tinjau pengaturan, kemudian pilih Generate token.
10. Salin token yang ditampilkan dan simpan di tempat aman. Token lengkap biasanya hanya ditampilkan saat dibuat; jika hilang, buat token baru.

5.4 Cara menggunakan PAT saat push

Contoh isian ketika Terminal meminta kredensial (teks di atas bukan perintah)

```
Username: USERNAME_GITHUB
Password: TEMPEL_PAT_DI_SINI
```

Pada prompt Password, tempel PAT lalu tekan Enter. Terminal tidak menampilkan tanda bintang atau karakter lain saat token ditempel. Jangan menekan Enter berkali-kali hanya karena layar tampak kosong.

5.5 Aturan keamanan PAT

- Jangan mengirim token melalui grup chat, email, media sosial, atau kepada teman satu kelompok.
- Jangan menuliskan PAT di dalam index.html, README.md, file konfigurasi proyek, atau dokumen yang akan diunggah.
- Jangan menampilkan PAT pada screenshot bukti praktik.

- Setiap anggota membuat dan menggunakan token miliknya sendiri; jangan menggunakan satu token bersama-sama.
- Simpan token di tempat yang aman dan terbatas aksesnya.
- Jika token hilang, dicurigai bocor, atau terlanjur dibagikan, segera revoke (cabut) token tersebut melalui Settings, lalu buat token pengganti.

ANALOGI KEAMANAN PAT seperti kunci akses pribadi. Orang yang memperoleh token dapat mencoba menggunakan hak akses yang diberikan kepada token tersebut. Karena itu, token harus diperlakukan sebagai rahasia seperti kata sandi.

5.6 Catatan untuk pembelajaran

Dokumentasi GitHub saat ini juga menyediakan fine-grained personal access token dan merekomendasikannya bila sesuai. Bahan ajar ini tetap menjelaskan alur classic PAT dengan scope repo karena itulah prosedur yang ditetapkan pada modul praktik. Pengajar dapat menyesuaikan instruksi apabila kebijakan akun atau tampilan GitHub yang digunakan di kelas berbeda.

6. MENYIAPKAN REPOSITORI LOKAL

Sebelum menghubungkan proyek ke GitHub, pastikan repositori lokal hasil pertemuan sebelumnya masih tersedia. Repositori lokal harus berada pada folder web-agensi-game, sudah diinisialisasi dengan Git, dan memiliki setidaknya satu commit.

6.1 Membuka Terminal dan masuk ke folder proyek

Buka Terminal pada VM Ubuntu. Gunakan perintah cd untuk berpindah ke folder proyek. Contoh berikut menganggap folder web-agensi-game berada di direktori kerja saat ini; sesuaikan lokasi folder jika proyek tersimpan di tempat lain.

```
cd web-agensi-game
pwd
ls -la
```

Perintah	Fungsi
cd web-agensi-game	Berpindah ke folder proyek web-agensi-game.
pwd	Menampilkan lokasi direktori aktif agar kita tahu sedang berada di folder mana.
ls -la	Menampilkan file dan folder, termasuk folder tersembunyi .git.

6.2 Memastikan folder adalah repositori Git

```
git status
git log --oneline
```

git status menunjukkan kondisi file dan branch saat ini. git log --oneline menampilkan riwayat commit secara ringkas. Apabila git log menampilkan setidaknya satu commit proyek sebelumnya, berarti terdapat riwayat yang dapat dikirim ke GitHub.

> **JIKA MUNCUL ERROR** Pesan fatal: not a git repository biasanya berarti Terminal tidak berada di dalam folder repositori Git. Periksa lokasi dengan pwd, kemudian gunakan cd untuk masuk ke folder web-agensi-game yang benar.

6.3 Mengapa kita memeriksa repositori lokal terlebih dahulu?

- Menghindari menjalankan perintah Git pada folder yang salah.
- Memastikan proyek yang akan dikirim adalah proyek yang benar.
- Memastikan sudah ada commit; tanpa commit, tidak ada riwayat proyek yang dapat di-push.
- Memastikan file yang akan diunggah sesuai dengan tugas dan tidak berisi rahasia.

7. MENGHUBUNGKAN REPOSITORI LOKAL KE GITHUB

7.1 Pengertian remote dan origin

Remote adalah nama rujukan untuk repositori yang berada di server. Dalam latihan ini, repositori GitHub menjadi remote dari repositori lokal. Nama origin lazim digunakan sebagai nama remote utama. Dengan menghubungkan origin, Git mengetahui alamat tujuan ketika kita menjalankan git push atau git pull.

7.2 Menyalin alamat HTTPS repositori

1. Buka halaman repositori web-agensi-game di GitHub.
2. Pilih tombol Code.
3. Pastikan tab HTTPS dipilih.
4. Salin alamat yang ditampilkan. Bentuk alamatnya adalah https://github.com/username/web-agensi-game.git.
5. Ganti username pada contoh dengan username GitHub milik sendiri. Jangan mengetik kata username secara literal.

7.3 Menambahkan remote origin

Perintah untuk mendaftarkan alamat remote

```
git remote add origin https://github.com/USERNAME_GITHUB/web-agensi-game.git
```

Ganti USERNAME_GITHUB dengan username akun sendiri. Perintah ini tidak mengunggah file. Perintah hanya menyimpan alamat repositori GitHub sebagai remote bernama origin.

7.4 Memeriksa remote

Perintah untuk melihat alamat remote

```
git remote -v
```

Apabila berhasil, Terminal menampilkan dua baris yang kurang lebih seperti berikut:

```
origin  https://github.com/USERNAME_GITHUB/web-agensi-game.git (fetch)
origin  https://github.com/USERNAME_GITHUB/web-agensi-game.git (push)
```

Keterangan fetch menunjukkan alamat yang digunakan saat mengambil informasi dari remote, sedangkan push menunjukkan alamat tujuan ketika mengirim commit ke remote. Keduanya biasanya sama pada latihan ini.

7.5 Jika origin sudah ada

Jika sebelumnya perintah git remote add origin pernah dijalankan, Git dapat menampilkan pesan fatal: remote origin already exists. Artinya, nama remote origin sudah terdaftar. Periksa alamatnya terlebih dahulu menggunakan git remote -v. Jika alamat salah, perbaiki dengan perintah berikut:

```
git remote set-url origin https://github.com/USERNAME_GITHUB/web-agensi-game.git
```

PERIKSA SEBELUM LANJUT Jangan menambahkan remote berkali-kali tanpa memeriksa pesan error. Gunakan git remote -v untuk mengetahui apakah origin sudah tersedia dan apakah alamatnya benar.

8. PUSH PERTAMA KE GITHUB

8.1 Apa yang dimaksud push?

Push adalah proses mengirim commit dari repositori lokal ke repositori remote. Setelah push berhasil, file dan riwayat commit dapat dilihat melalui halaman repositori di GitHub. Push pertama pada latihan ini dilakukan dengan mengunggah branch main.

8.2 Mengubah nama branch menjadi main

```
git branch -M main
```

Branch adalah jalur pengembangan proyek. Perintah tersebut mengubah nama branch yang sedang aktif menjadi main. Nama main digunakan sebagai branch utama dalam latihan ini.

8.3 Mengirim commit pertama

```
git push -u origin main
```

Arti bagian perintah:

Bagian	Makna
git push	Memerintahkan Git mengirim commit ke remote.
-u	Mengatur hubungan upstream/tracking agar branch lokal mengetahui branch remote yang diikuti.
origin	Nama remote tujuan yang telah dibuat sebelumnya.

Bagian	Makna
main	Nama branch yang dikirim.

8.4 Memasukkan kredensial

1. Jika Terminal meminta Username, masukkan username GitHub.
2. Jika Terminal meminta Password, tempel Personal Access Token (PAT), bukan password akun GitHub.
3. Karakter token tidak terlihat saat diketik atau ditempel. Kondisi ini normal.
4. Tunggu sampai proses selesai. Jangan menutup Terminal ketika proses push masih berjalan.

8.5 Memeriksa keberhasilan push

1. Buka browser dan masuk ke halaman repositori web-agensi-game.
2. Muat ulang (refresh) halaman repositori.
3. Pastikan file index.html muncul pada daftar file.
4. Buka bagian riwayat commit dan pastikan commit yang berasal dari repositori lokal terlihat.
5. Bandingkan nama file dan pesan commit dengan kondisi proyek lokal.

TANDA KEBERHASILAN Push pertama dianggap berhasil apabila git push selesai tanpa error dan halaman GitHub menampilkan file index.html beserta riwayat commit proyek.

9. MELAKUKAN PERUBAHAN DAN PUSH LANJUTAN

Setelah push pertama berhasil, perubahan berikutnya dapat dikirim dengan urutan add, commit, dan push. Perintah -u origin main biasanya tidak perlu ditulis lagi karena hubungan upstream sudah ditetapkan pada push pertama.

9.1 Alur perubahan file

Langkah	Perintah / aktivitas	Penjelasan
1. Edit	Ubah file proyek, misalnya index.html.	Buat perubahan yang nyata dan sesuai tujuan proyek.
2. Add	git add .	Memasukkan perubahan pada folder kerja ke Staging Area.
3. Commit	git commit -m "Menambahkan bagian layanan agensi"	Menyimpan perubahan yang sudah di-stage ke riwayat lokal dengan pesan yang jelas.
4. Push	git push	Mengirim commit terbaru dari lokal ke GitHub.

9.2 Memahami Staging Area

Staging Area adalah tempat persiapan sebelum perubahan disimpan sebagai commit. Perintah git add . memilih perubahan pada direktori kerja saat ini untuk dimasukkan ke staging. Setelah itu, git commit membuat catatan perubahan berdasarkan isi staging tersebut.

9.3 Praktik contoh

Contoh: tambahkan bagian layanan agensi pada file index.html. Simpan perubahan, kemudian jalankan perintah berikut dari folder repositori:

```
git status
git add .
git status
git commit -m "Menambahkan bagian layanan agensi"
git push
```

Periksa kembali halaman repositori GitHub. Commit dengan pesan Menambahkan bagian layanan agensi seharusnya terlihat pada riwayat commit. Jika belum terlihat, periksa apakah perintah git push berhasil dan apakah remote yang digunakan benar.

9.4 Mengapa pesan commit harus jelas?

Pesan commit menjadi keterangan singkat tentang perubahan yang disimpan. Pesan seperti "update" atau "coba" kurang informatif. Pesan seperti "Menambahkan bagian layanan agensi" membantu anggota tim memahami isi perubahan tanpa harus langsung membuka seluruh file.

Pesan kurang jelas	Pesan lebih jelas
update	Memperbaiki teks pada halaman beranda
perubahan	Menambahkan bagian layanan agensi
fix	Memperbaiki tautan navigasi pada index.html

10. MENAMBAHKAN KOLABORATOR TIM

10.1 Pengertian kolaborator

Kolaborator adalah pengguna GitHub yang diberi izin untuk mengakses dan berkontribusi pada repositori. Dengan menambahkan anggota kelompok sebagai kolaborator, pemilik repositori memberi akses agar anggota dapat bekerja pada proyek yang sama sesuai izin yang diberikan.

> **PENGORGANISASIAN PRAKTIK** Setiap peserta didik dapat mencoba pembuatan akun dan repositori secara mandiri. Untuk kerja kelompok, sepakati satu repositori utama milik pemilik/ketua kelompok sebagai pusat kerja bersama. Pastikan semua anggota mengetahui username pemilik dan alamat repositori utama tersebut.

10.2 Langkah pemilik repositori

1. Pastikan akun anggota kelompok sudah memiliki username GitHub.

2. Buka halaman repositori web-agensi-game yang dimiliki kelompok.
3. Pilih tab Settings pada halaman repositori.
4. Pada bagian Access, pilih Collaborators. Nama menu dapat sedikit berbeda sesuai tampilan GitHub.
5. Pilih Add people.
6. Ketik username GitHub anggota kelompok yang akan diundang.
7. Pilih akun yang benar dari daftar hasil pencarian.
8. Konfirmasikan dengan tombol Add [nama] to repository atau tombol setara yang muncul.
9. Pastikan undangan telah terkirim.

10.3 Langkah anggota yang diundang

1. Buka email yang terhubung dengan akun GitHub atau periksa notifikasi GitHub.
2. Cari undangan untuk berkolaborasi pada repositori web-agensi-game.
3. Buka undangan dan periksa nama repositori serta pemiliknya.
4. Pilih Accept invitation untuk menerima undangan.
5. Buka kembali repositori dan pastikan akses sudah diberikan.

10.4 Memastikan remote anggota mengarah ke repo kelompok

Agar anggota benar-benar bekerja pada repositori kelompok yang sama, alamat remote origin pada komputer setiap anggota harus menunjuk ke repositori milik pemilik/ketua kelompok. Ini merupakan penjelasan teknis tambahan untuk memperjelas alur kolaborasi. Jika setiap anggota sebelumnya membuat repositori pribadi saat latihan, pastikan origin tidak tetap mengarah ke repositori pribadi ketika mulai bekerja pada repo kelompok.

Pertama, pemilik kelompok membagikan username GitHub dan alamat repositori kelompok. Anggota membuka Terminal pada folder proyek yang akan digunakan, lalu memeriksa remote:

```
git remote -v
```

Jika origin masih mengarah ke repositori pribadi, ubah alamatnya menjadi alamat repositori milik pemilik kelompok:

```
git remote set-url origin https://github.com/USERNAME_PEMILIK/web-agensi-game.git
```

Ganti USERNAME_PEMILIK dengan username GitHub pemilik repositori kelompok. Setelah itu, periksa kembali:

```
git remote -v
```

Jika belum ada remote bernama origin, gunakan git remote add origin dengan URL repositori kelompok. Pastikan seluruh anggota melihat URL yang sama sebelum melanjutkan. Setelah remote benar dan undangan diterima, jalankan git pull untuk mengambil kondisi terbaru. Jika muncul pesan riwayat tidak cocok atau konflik yang belum dipahami, hentikan proses dan minta arahan guru; jangan memaksakan push.

PEMBAGIAN AKSES Setiap anggota menggunakan akun dan token masing-masing. Jangan meminta atau menggunakan PAT milik teman. Akses kolaborasi diberikan melalui undangan repositori, bukan dengan membagi token.

11. ALUR KERJA KOLABORASI YANG AMAN

11.1 Urutan utama: PULL - EDIT - ADD - COMMIT - PUSH

Urutan	Perintah / tindakan	Tujuan
1. PULL	git pull	Mengambil dan menggabungkan perubahan terbaru dari GitHub sebelum mulai bekerja.
2. EDIT	Mengubah file proyek.	Mengerjakan fitur atau bagian website yang menjadi tanggung jawab.
3. ADD	git add .	Memilih perubahan yang akan dimasukkan ke commit.
4. COMMIT	git commit -m "pesan"	Menyimpan perubahan ke riwayat repositori lokal.
5. PUSH	git push	Mengirim commit lokal ke repositori GitHub.

Biasakan memulai setiap sesi kerja dengan git pull. Dengan cara tersebut, anggota tim bekerja berdasarkan kondisi kode yang lebih baru. Setelah selesai mengubah file, simpan perubahan melalui add dan commit, lalu push agar anggota lain dapat mengambilnya.

11.2 Contoh kegiatan kolaborasi

Sebuah kelompok terdiri atas Wulan, Fajar, dan Siti. Wulan bertugas memperbaiki tampilan beranda, Fajar membuat bagian layanan, dan Siti memeriksa tautan navigasi. Sebelum mulai, setiap anggota menjalankan git pull. Mereka berkomunikasi mengenai file yang dikerjakan agar tidak mengubah bagian yang sama tanpa koordinasi. Setelah perubahan selesai, setiap anggota menjalankan git add, git commit, dan git push.

11.3 Konflik dan penolakan push

Jika repositori GitHub telah menerima commit baru dari anggota lain, sedangkan repositori lokal belum mengambil perubahan tersebut, Git dapat menolak push. Penolakan ini melindungi riwayat remote agar perubahan yang sudah ada tidak tertimpa begitu saja.

Solusi awal sesuai alur pada modul adalah menjalankan git pull terlebih dahulu, membaca pesan yang muncul, menyelesaikan konflik jika ada, kemudian mengulangi git push. Jika muncul penanda konflik atau peserta didik tidak memahami perubahan yang harus dipertahankan, hentikan proses dan minta bantuan guru. Jangan menggunakan perintah force push untuk mengatasi penolakan pada latihan ini.

11.4 Studi kasus: Kelompok Wulan

KASUS Kelompok Wulan sudah berhasil mengunggah kode web agensi ke GitHub. Keesokan

harinya, Fajar langsung mengedit index.html tanpa menjalankan git pull. Ketika melakukan git push, proses ditolak karena GitHub sudah berisi perubahan Wulan. Fajar juga berniat membagikan PAT miliknya ke grup chat agar semua anggota memakai token yang sama.

Pertanyaan diskusi dan pembahasan

1. Mengapa push Fajar gagal? Apa yang seharusnya dilakukan?

Push gagal karena versi di GitHub lebih baru daripada versi lokal Fajar. Git menolak push agar perubahan Wulan tidak tertimpa. Sebelum mengedit, Fajar seharusnya menjalankan git pull untuk menyinkronkan repositori lokal dengan perubahan terbaru.

2. Apakah membagikan PAT ke grup chat aman?

Tidak aman. PAT adalah kredensial rahasia. Siapa pun yang memiliki token dapat mencoba mengakses sumber daya sesuai izin token tersebut. Setiap anggota harus membuat token sendiri. Jika token sudah terlanjur dibagikan, pemiliknya harus segera revoke token itu dan membuat token baru.

3. Apa urutan kerja yang benar?

Sebelum mengedit: git pull. Setelah mengedit: git add . lalu git commit -m "pesan yang jelas" dan git push. Anggota juga perlu berkomunikasi mengenai file atau fitur yang dikerjakan agar perubahan tidak saling berbenturan.

12. PEMECAHAN MASALAH (TROUBLESHOOTING)

Pesan error adalah petunjuk untuk mengetahui bagian yang perlu diperiksa. Baca pesan secara teliti, periksa lokasi folder, alamat remote, branch, dan autentikasi. Jangan langsung mengulang semua perintah tanpa memahami penyebabnya.

Pesan error	Kemungkinan penyebab	Cara mengatasi
fatal: remote origin already exists.	Remote bernama origin sudah pernah ditambahkan.	Jalankan git remote -v. Jika alamat salah, gunakan git remote set-url origin https://github.com/USERNAME_GITHUB/web-agensi-game.git.
Authentication failed / password authentication was removed	Password akun GitHub dimasukkan sebagai pengganti token atau PAT tidak memiliki izin yang sesuai.	Ulangi autentikasi. Masukkan username GitHub dan PAT pada prompt Password. Pastikan token sesuai instruksi dan scope/izin yang diperlukan tersedia.
Repository not found	Username atau nama repo pada URL salah, atau repositori tidak tersedia/akses belum diberikan.	Periksa alamat dengan git remote -v. Pastikan username, nama repo, dan hak akses benar. Perbaiki URL menggunakan git remote set-url origin <url-yang-benar>.

Pesan error	Kemungkinan penyebab	Cara mengatasi
failed to push some refs ... fetch first	Remote memiliki commit yang belum ada di lokal, misalnya perubahan dari anggota tim.	Jalankan git pull terlebih dahulu. Periksa apakah ada konflik, selesaikan jika ada, lalu jalankan git push.
src refspec main does not match any	Belum ada commit yang dapat dikirim atau branch lokal bukan bernama main.	Periksa git log --oneline. Buat commit jika belum ada, lalu jalankan git branch -M main.
fatal: not a git repository	Terminal tidak berada di dalam folder repositori Git.	Periksa lokasi dengan pwd. Masuk ke folder web-agensi-game menggunakan cd, lalu jalankan git status.

12.1 Jika repositori GitHub terlanjur dibuat dengan README

Pada latihan ini, repositori remote seharusnya dibuat tanpa README agar repositori baru kosong. Jika peserta didik terlanjur mencentang Add a README file, remote telah memiliki commit awal yang berbeda dari riwayat lokal. Sesuai catatan guru pada modul, salah satu cara menggabungkan riwayat tersebut adalah:

```
git pull origin main --allow-unrelated-histories
# Periksa dan selesaikan konflik jika muncul
git push -u origin main
```

PERHATIAN GURU Perintah --allow-unrelated-histories hanya digunakan pada kasus riwayat lokal dan remote yang dibuat terpisah. Pastikan peserta didik memahami pesan merge dan konflik yang mungkin muncul. Untuk pemula, alternatif yang lebih sederhana adalah membuat ulang repositori remote yang kosong sesuai instruksi latihan. Jangan menghapus atau menimpa pekerjaan tanpa pemeriksaan.

13. REFERENSI PERINTAH GIT PADA PERTEMUAN INI

Perintah	Fungsi	Kapan digunakan
pwd	Melihat direktori aktif.	Memastikan Terminal berada di folder proyek.
ls -la	Melihat file dan folder termasuk yang tersembunyi.	Memeriksa isi folder proyek.
git status	Melihat status file, staging, dan branch.	Memeriksa keadaan sebelum dan sesudah perubahan.

Perintah	Fungsi	Kapan digunakan
git log --oneline	Melihat riwayat commit secara ringkas.	Memastikan commit sudah ada.
git remote add origin <url>	Menghubungkan repo lokal dengan remote GitHub bernama origin.	Saat menghubungkan repo untuk pertama kali.
git remote -v	Melihat alamat remote untuk fetch dan push.	Memverifikasi alamat GitHub.
git remote set-url origin <url>	Mengubah alamat remote origin.	Jika URL origin salah atau perlu diperbaiki.
git branch -M main	Mengubah nama branch aktif menjadi main.	Sebelum push pertama pada latihan.
git push -u origin main	Mengirim branch main untuk pertama kali dan menetapkan upstream.	Push pertama.
git add .	Memasukkan perubahan pada direktori ke staging area.	Setelah mengedit file.
git commit -m "pesan"	Menyimpan perubahan ke riwayat lokal dengan pesan.	Setelah git add.
git push	Mengirim commit terbaru ke remote.	Setelah push awal dan setelah commit baru.
git pull	Mengambil dan menggabungkan perubahan terbaru dari remote.	Sebelum mulai bekerja bersama tim.

PERHATIKAN TANDA KURUNG SIKU Pada contoh perintah, <url> berarti alamat repositori yang harus diganti dengan URL sebenarnya. Jangan mengetik tanda < dan > sebagai bagian dari alamat. Ganti USERNAME_GITHUB dengan username akun sendiri.

14. PRAKTIK TERBIMBING DAN CEK HASIL

14.1 Target praktik

Pada akhir praktik, setiap peserta didik diharapkan memiliki akun GitHub aktif, repositori Public bernama web-agensi-game tanpa README bawaan, PAT yang disimpan dengan aman, remote origin yang benar, push pertama yang berhasil, satu push lanjutan, dan bukti bahwa anggota kelompok telah diundang sebagai kolaborator.

14.2 Checklist praktik

No.	Langkah / bukti yang harus tersedia	Selesai

No.	Langkah / bukti yang harus tersedia	Selesai
1	Akun GitHub aktif dan email terverifikasi.	☐
2	Repositori web-agensi-game dibuat dengan visibility Public.	☐
3	Repositori dibuat tanpa mencentang Add a README file.	☐
4	PAT berhasil dibuat dan disimpan aman; nilai token tidak ditampilkan.	☐
5	git remote -v menampilkan alamat origin yang benar.	☐
6	git branch -M main dan git push -u origin main berhasil.	☐
7	File index.html dan riwayat commit terlihat di GitHub.	☐
8	Perubahan lanjutan di-commit dan di-push ke GitHub.	☐
9	Anggota kelompok ditambahkan sebagai kolaborator dan undangan diterima.	☐

14.3 Bukti screenshot yang dikumpulkan

- Tampilan profil akun GitHub yang aktif.
- Tampilan repositori web-agensi-game yang telah dibuat.
- Tampilan daftar token tanpa menampilkan nilai token.
- Hasil perintah git remote -v.
- Bukti push pertama berhasil dan file index.html tampak pada GitHub.
- Riwayat commit yang menunjukkan perubahan lanjutan.
- Tampilan pengaturan Collaborators yang menunjukkan anggota telah diundang.

> **KEAMANAN BUKTI** Periksa setiap screenshot sebelum dikumpulkan. Pastikan tidak ada PAT, password, kode pemulihan, atau informasi akun rahasia yang terlihat.

14.4 Refleksi singkat

- Bagian mana dari materi GitHub yang paling mudah kamu pahami?
- Bagian mana yang masih sulit: membuat token, menghubungkan remote, push, atau menambahkan kolaborator?
- Mengapa kode yang sudah di-commit belum tentu terlihat di GitHub?
- Apa yang akan kamu lakukan agar kerja kelompok di GitHub lebih aman dan teratur?

15. LATIHAN PEMAHAMAN

A. Pilihan Ganda

1. Perbedaan mendasar antara Git dan GitHub adalah ...

A. Git dan GitHub merupakan aplikasi yang sama.

B. Git mencatat riwayat perubahan, sedangkan GitHub menyimpan repositori Git secara online.

C. GitHub hanya bisa digunakan tanpa Git.

D. Git hanya digunakan untuk membuat desain gambar.

Jawaban: _____

2. Perintah untuk menghubungkan repositori lokal dengan GitHub adalah ...

A. git connect origin

B. git remote add origin <url>

C. git link github

D. git push origin

Jawaban: _____

3. Saat Terminal meminta Password ketika melakukan push melalui HTTPS, yang dimasukkan adalah ...

A. Password akun GitHub biasa

B. Nama repositori

C. Personal Access Token

D. Alamat email

Jawaban: _____

4. Perintah untuk mengambil dan menggabungkan perubahan terbaru dari GitHub adalah ...

A. git pull

B. git push

C. git add .

D. git commit

Jawaban: _____

5. Tindakan yang tidak aman terkait PAT adalah ...

A. Menyimpannya di tempat aman

B. Membuat token sendiri untuk setiap anggota

C. Membagikan token ke grup chat

D. Revoke token yang diduga bocor

Jawaban: _____

B. Esai Singkat

1. Mengapa setiap anggota kelompok sebaiknya menjalankan git pull sebelum mulai mengedit proyek?

.....

.....

2. Mengapa repositori GitHub perlu aktif dan berisi kode proyek sebelum website dipublikasikan melalui GitHub Pages?

.....

.....

3. Jelaskan dengan urut proses mengirim perubahan website dari komputer lokal sampai terlihat di GitHub.

.....
.....

4. Mengapa setiap anggota harus memiliki PAT sendiri dan tidak boleh membagikannya kepada teman?

.....
.....

16. RANGKUMAN DAN KUNCI JAWABAN

16.1 Rangkuman

- Git adalah tools version control yang mencatat riwayat perubahan proyek pada repositori lokal.
- GitHub adalah platform online untuk menyimpan repositori Git, berbagi proyek, dan bekerja sama.
- Repositori lokal berada di komputer/VM, sedangkan repositori remote pada latihan ini berada di GitHub.
- Perintah git remote add origin <url> menghubungkan repositori lokal dengan GitHub; git remote -v memeriksa alamatnya.
- Push pertama dilakukan dengan git branch -M main lalu git push -u origin main.
- Perubahan lanjutan dilakukan dengan git add ., git commit -m "pesan", lalu git push.
- Setiap anggota yang diundang harus menerima undangan sebagai kolaborator menggunakan akun GitHub sendiri.
- Alur kerja aman yang dibiasakan adalah git pull sebelum bekerja, kemudian edit, add, commit, dan push.
- PAT adalah rahasia. Jangan membagikan, menampilkan, atau memasukkannya ke dalam file proyek. Revoke token jika hilang atau bocor.

16.2 Kunci pilihan ganda

Nomor	Kunci	Alasan
1	B	Git mencatat riwayat perubahan; GitHub menyimpan repositori secara online.
2	B	git remote add origin <url> menambahkan remote bernama origin.
3	C	PAT digunakan sebagai pengganti password akun saat operasi Git melalui HTTPS.
4	A	git pull mengambil dan menggabungkan perubahan terbaru dari remote.
5	C	Token tidak boleh dibagikan karena bersifat rahasia.

16.3 Pedoman jawaban esai

Nomor	Pokok jawaban yang diharapkan

Nomor	Pokok jawaban yang diharapkan
1	git pull menyinkronkan kode lokal dengan perubahan terbaru dari anggota tim, membantu mencegah push ditolak, konflik, atau perubahan teman tertimpa.
2	GitHub Pages menggunakan sumber dari repositori GitHub. Kode proyek harus diunggah terlebih dahulu agar dapat dipublikasikan.
3	Edit file, git add ., git commit -m "pesan yang jelas", lalu git push. Sebelum memulai pekerjaan bersama tim, jalankan git pull.
4	PAT adalah kredensial pribadi. Orang yang memperoleh token dapat menggunakan hak akses token. Setiap anggota harus menggunakan token sendiri dan mencabut token yang bocor.

GLOSARIUM

Istilah	Arti
Add	Memasukkan perubahan ke Staging Area sebelum dibuat menjadi commit.
Branch	Cabang/jalur pengembangan dalam repositori Git.
Cloud	Layanan komputasi/penyimpanan yang dapat diakses melalui internet.
Commit	Catatan perubahan yang disimpan dalam riwayat repositori Git.
Collaborator	Pengguna yang diberi akses untuk berkontribusi pada repositori.
Git	Perangkat lunak version control untuk mencatat dan mengelola perubahan file proyek.
GitHub	Platform online untuk hosting repositori Git dan mendukung kolaborasi.
Local repository	Repositori Git yang berada pada komputer atau VM sendiri.
PAT	Personal Access Token; token pribadi untuk autentikasi dan akses sesuai izin yang diberikan.
Push	Mengirim commit dari repositori lokal ke remote.
Pull	Mengambil dan menggabungkan perubahan remote ke repositori lokal.
Remote repository	Repositori yang berada di layanan/server online.
Staging Area	Area persiapan untuk memilih perubahan yang akan disimpan dalam commit.
URL	Alamat yang menunjukkan lokasi repositori atau sumber daya di internet.